

Density Vector

#sql-server #performance #statistics #optimizer

What the Density Vector Is

Every statistics object in SQL Server has two components:

1. The Histogram. Up to 200 steps showing the distribution of values: how many rows have value X, how many fall between X and Y, and so on. This is what the optimizer uses when it knows the exact value being searched (when it can sniff a parameter).
2. The Density Vector. A single number per column (or column combination) representing $1 / \text{CountOfDistinctValues}$. This is what the optimizer uses when it doesn't know the search value, for example when you use **local variables** instead of parameters.

```
-- Density for a column with 1,000 distinct values:
-- Density = 1/1000 = 0.001
-- Estimated rows = TotalRows * Density = 500,000 * 0.001 = 500 rows
```

Why It Matters

When you mitigate **Parameter Sniffing** by assigning parameters to local variables, you're trading the histogram (precise, value-specific estimates) for the density vector (average-case estimates). The optimizer computes:

$\text{EstimatedRows} = \text{TotalRowsInTable} \times \text{Density}$

This is the average number of rows per distinct value. For uniformly distributed data, this is accurate. For skewed data, it can be wildly wrong.

The Skew Problem

Consider a ShipToteId column where:

- Most totes have 2 to 5 shipments.
- A few mega-totes have 500 or more shipments.
- 370 distinct tote values across 1,300 rows.
- $\text{Density} = 1/370 = 0.0027$.
- $\text{Estimated rows} = 1,300 \times 0.0027$, which is about 3.5 rows.

The optimizer builds a plan expecting about 4 rows (nested loop, small memory grant). If the actual call hits a mega-tote with 500 rows, the plan is catastrophically wrong. This time it's wrong on every call, not just some calls. You've traded intermittent bad plans (sniffing) for a consistently mediocre plan (density).

Measuring Skew

To decide whether density-based estimation is safe, analyze the data distribution:

```
;With Aggs As (
  Select
    Count(*)           As TotalRows,
    Count(Distinct ShipToteId) As DistinctVals,
    Min(Cnt)           As MinPerVal,
    Max(Cnt)           As MaxPerVal,
    Avg(Cnt)           As AvgPerVal,
    Stdev(Cnt)          As StdevPerVal
```

```

    From (
        Select ShipToteId, Count(*) As Cnt
        From SrtShipToteShipmentAssoc With (NoLock)
        Group By ShipToteId
    ) Grouped
),
Pctls As (
    Select Distinct
        Percentile_Cont(0.50) Within Group (Order By Cnt) Over() As P50,
        Percentile_Cont(0.90) Within Group (Order By Cnt) Over() As P90,
        Percentile_Cont(0.95) Within Group (Order By Cnt) Over() As P95,
        Percentile_Cont(0.99) Within Group (Order By Cnt) Over() As P99
    From (
        Select ShipToteId, Count(*) As Cnt
        From SrtShipToteShipmentAssoc With (NoLock)
        Group By ShipToteId
    ) Grouped
)
Select
    A.TotalRows, A.DistinctVals,
    A.MinPerVal, A.MaxPerVal, A.AvgPerVal, A.StdevPerVal,
    Cast(A.MaxPerVal As Float) / NullIf(A.AvgPerVal, 0) As Max_to_Mean_Ratio,
    P.P50, P.P90, P.P95, P.P99
From Aggs A
Cross Join Pctls P

```

Interpreting the Results

Metric	Healthy	Concerning	Dangerous
Max_to_Mean_Ratio	< 5x	5 to 20x	> 20x
P99 / P50	< 3x	3 to 10x	> 10x
Stdev / Mean (CV)	< 1.0	1.0 to 3.0	> 3.0

If the data is healthy (low skew), density-based estimation is safe and the local variable mitigation works well. If the data is skewed, you may need OPTION (RECOMPILE) or plan guides instead.

Bucket Histogram

For a visual understanding of the distribution:

```

Select
    Case
        When Cnt = 1 Then '1'
        When Cnt Between 2 And 3 Then '2-3'
        When Cnt Between 4 And 5 Then '4-5'
        When Cnt Between 6 And 10 Then '6-10'
        When Cnt Between 11 And 25 Then '11-25'
        When Cnt Between 26 And 50 Then '26-50'
        Else '51+'
    End As Bucket,
    Count(*) As ValuesInBucket,
    Cast(Count(*) * 100.0 / Sum(Count(*)) Over() As Decimal(5,2)) As PctOfTotal

```

```

From (
  Select ShipToteId, Count(*) As Cnt
  From SrtShipToteShipmentAssoc With (NoLock)
  Group By ShipToteId
) G
Group By
  Case
    When Cnt = 1 Then '1'
    When Cnt Between 2 And 3 Then '2-3'
    When Cnt Between 4 And 5 Then '4-5'
    When Cnt Between 6 And 10 Then '6-10'
    When Cnt Between 11 And 25 Then '11-25'
    When Cnt Between 26 And 50 Then '26-50'
    Else '51+'
  End
Order By Min(Cnt)

```

Viewing the Actual Density Vector

```

DBCC SHOW_STATISTICS ('SrtShipToteShipmentAssoc', 'IX_ShipToteId')
-- Result set 2 is the density vector
-- Result set 3 is the histogram

```

The density vector shows one row per column combination in the statistics object, with the density value and average row length.

Decision Framework

```

Is the proc called > 1000 times/day?
├─ No → OPTION (RECOMPILE) is safe. Use it.
├─ Yes → Check data skew:
│   └─ Max_to_Mean < 5x → Local variables are safe.
│       Density estimate is "good enough."
│   └─ Max_to_Mean > 5x → Consider:
│       └─ Plan guides for specific problem values
│       └─ OPTIMIZE FOR UNKNOWN (same as density, but explicit)
│       └─ Query redesign (eliminate the parameter sensitivity)

```

Real-World Example

For `lsp_SrtGetShipTotelfExists`, Francis reported **Parameter Sniffing** as the root cause of plan instability. I ran the skew analysis on `SrtShipToteShipmentAssoc` and found a `Max_to_Mean_Ratio` of about 2x with a tight distribution (75% of values in the 26 to 50 bucket). This confirmed the density vector estimate would be accurate. The data simply isn't skewed enough for sniffing to cause the observed instability. The real issue turned out to be forced plan failures visible in **Query Store Triage**.

Related Concepts

- **Parameter Sniffing**: the density vector is the optimizer's fallback when sniffing is prevented.
- **Query Store Triage**: use Query Store to validate whether the density-based plan performs consistently.
- **Table Variables vs Temp Tables**: table variables essentially have density of "1 row always".